PNY Trainings — Cyber Security Department

Web Application Security — Final Project Assignments

Project 5: Mini Penetration Test Report

Full-Scope Assessment of OWASP Juice Shop v20.3.0

Student: Hamza Iqbal

Environment: Local lab — Docker on Windows 11 (own host)

Classification: Lab exercise — intentionally vulnerable training application

Table of Contents

1. Executive Summary
2. Scope & Rules of Engagement
3. Methodology & Tools
4. Reconnaissance Summary
5. Findings
6. Findings Summary & Attack Path
7. Remediation Roadmap
8. Conclusion
Appendix A — Live Demo Runbook (10 minutes)
Appendix B — Evidence Index

1. Executive Summary

This report presents a full penetration test of OWASP Juice Shop v20.3.0, conducted in an isolated local lab as the capstone of the Web Application Security final projects. The assessment followed the same path as the preceding projects — reconnaissance (Project 1), injection testing (Project 2), access-control auditing (Project 3), and scanner-based analysis with a hardening exercise (Project 4) — and consolidates them into a single engagement with two additional findings.

Overall risk posture: CRITICAL. Seven findings were confirmed — 1 Critical, 4 High, 2 Medium. An unauthenticated attacker can bypass the login entirely and become administrator (F1); a registered user with no privileges can read and modify other customers' baskets (F3) and download the shop's internal configuration (F4); and without any account at all, an attacker can browse a directory containing a password vault and confidential business documents (F5). These are not theoretical weaknesses: every finding in Section 5 was exploited or retrieved live in the lab, and each is supported by screenshot evidence.

The single most important theme is that the application repeatedly trusts the client: it trusts input inside SQL queries (F1), trusts markup inside the DOM (F2), trusts object identifiers from the browser (F3), and trusts that hiding an interface is the same as protecting it (F4, F5, F6). The remediation roadmap in Section 7 is organized around removing that misplaced trust.

2. Scope & Rules of Engagement

- **In scope —** OWASP Juice Shop v20.3.0 (Docker image bkimminich/juice-shop) at http://localhost:3000, plus the hardened proxy instance at http://localhost:4000 used for remediation validation.
- **Accounts —** The tester's own accounts (attacker@test.com, a victim test account, and the built-in administrator obtained via F1) on the local instance only.
- **Boundary —** Isolated lab on the tester's own Windows 11 host. No external system, network, or person was tested — consistent with the lab-only scope of the assignment and with PECA 2016.
- **Constraints —** No denial of service, no persistence mechanisms, no data destruction; the container can be reset at any time.

3. Methodology & Tools

The assessment followed a standard web penetration-testing flow aligned with the OWASP Web Security Testing Guide: (1) reconnaissance and attack-surface mapping; (2) automated scanning to establish a baseline; (3) manual verification of scanner results; (4) targeted exploitation of the confirmed weaknesses; (5) post-exploitation review of what each compromise enables; and (6) reporting with remediations, including a remediation-validation exercise in which the header-class findings were fixed and rescanned.

Tool	Version	Use in this engagement

OWASP ZAP	2.17.0	Spidering/attack-surface mapping; baseline scan (8 alerts) and post-hardening rescan
Google Chrome + DevTools	155	Manual exploitation, session-storage tampering, header verification
Docker Desktop	4.95.0	Target hosting; nginx hardening proxy
nginx (Docker image)	latest	Remediation validation — security-header enforcement (Project 4)

4. Reconnaissance Summary

Reconnaissance (detailed in the Project 1 report) identified an Angular single-page application backed by Node.js/Express with a SQLite database (application version 20.3.0 disclosed without authentication). ZAP spidering mapped 21 endpoints, including /api/ resources (Challenges, Feedbacks, Quantitys, SecurityQuestions) and /rest/ services (products, user, admin, captcha, memories), plus a browsable /ftp directory and /robots.txt. Fifteen or more user-controlled input parameters were catalogued; the login form, the search box, and the basket identifier — the inputs behind findings F1, F2, and F3 — were all visible already at this stage, which is why the mapping phase drives the rest of this report.

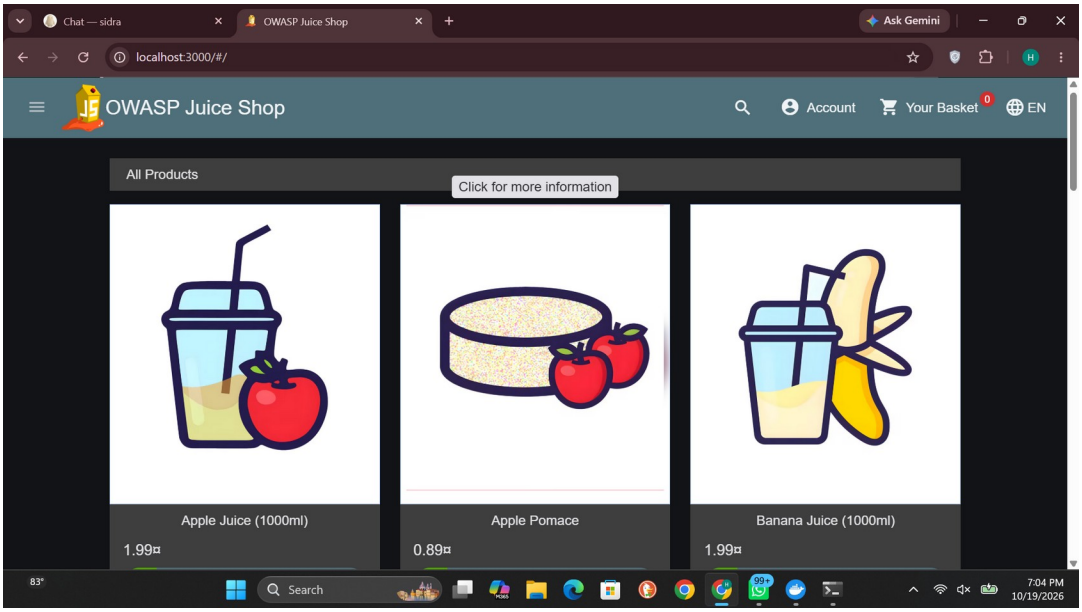


Figure 1 — The target application (product catalogue) as mapped during reconnaissance

5. Findings

5.1 Finding 1 — SQL Injection — Authentication Bypass

Severity: Critical

CWE: CWE-89 — Improper Neutralization of Special Elements used in an SQL Command

Location: Login form (/#/login) → POST /rest/user/login

The login query concatenates the submitted email directly into SQL. Submitting the payload ' or 1=1-- as the email (with any password) makes the WHERE clause always true and comments out the password check, so the database returns its first user — the administrator. The tester was authenticated with full administrative rights without knowing any credential ("Login Admin" challenge solved).

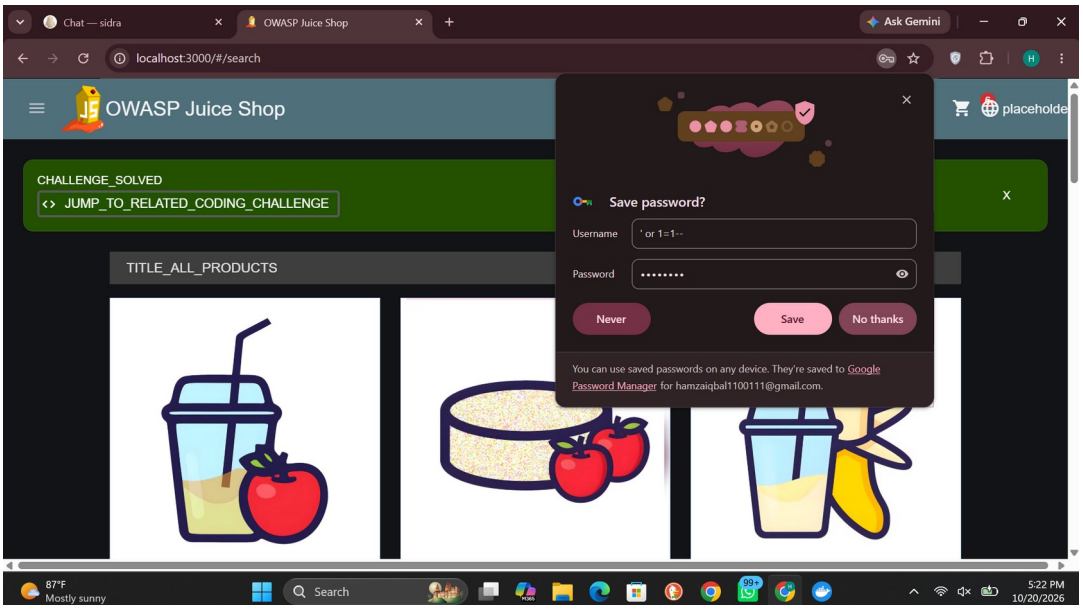


Figure 2 — F1: the payload (' or 1=1--) submitted as the username; CHALLENGE_SOLVED confirms administrator authentication

Impact: Complete authentication bypass and account takeover of the administrator; the same injection point supports UNION/error-based extraction of the entire database, including password hashes and personal data. No prior access is required.

Remediation: Use parameterized queries/prepared statements for all database access (ORM bindings), validate input server-side as defence in depth, and run the application with a least-privilege database account.

5.2 Finding 2 — DOM-Based Cross-Site Scripting (XSS)

Severity: High

CWE: CWE-79 — Improper Neutralization of Input During Web Page Generation

Location: Product search (/#/search?q=)

The search term is inserted into the DOM without adequate sanitization. Plain <script> is stripped, but an iframe with a javascript: source — <iframe src="javascript:alert('xss')"> — bypasses the filtering and executes in the victim's browser, as proven by the executed alert and the solved "DOM XSS" challenge.

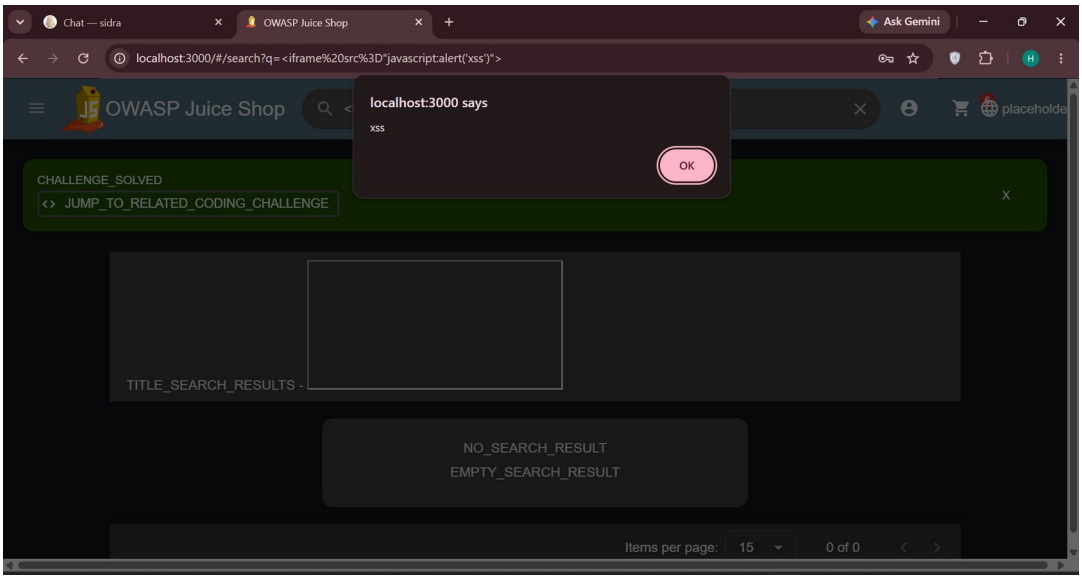


Figure 3 — F2: injected iframe executes alert('xss'); the payload is visible in the URL and the challenge is solved

Impact: Script execution in a victim's session enables session/token theft, credential-phishing overlays, and actions performed as the victim; the payload is deliverable as an ordinary-looking shop link.

Remediation: Render user input as text, never as HTML; rely on Angular's default output encoding and never call bypassSecurityTrust* on user data; deploy a restrictive Content-Security-Policy as defence in depth.

5.3 Finding 3 — Insecure Direct Object Reference — Another User's Basket

Severity: High

CWE: CWE-639 — Authorization Bypass Through User-Controlled Key

Location: GET /rest/basket/:id — basket identifier (bid) held in session storage

The basket is selected by a client-held identifier (bid). The tester's own bid was 6 and empty; a second (victim) account's was 7 with two items. Editing the stored bid from 6 to 7 and reloading displayed — and allowed editing — the victim's basket under the attacker's session ("View Basket" challenge solved). The server never checks that the basket belongs to the caller.

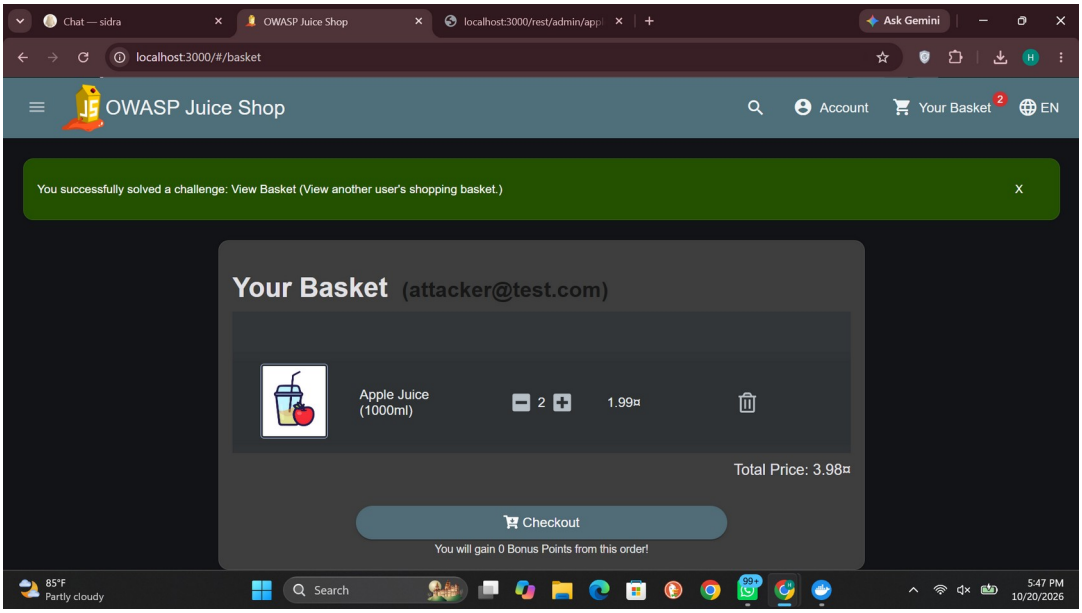


Figure 4 — F3: the victim's basket (2 × Apple Juice, total 3.98) displayed under the attacker's account, attacker@test.com

Impact: Any registered user can read, modify, delete, or check out any other user's basket by incrementing a small integer — a confidentiality and integrity breach across the whole customer base.

Remediation: Verify object ownership on the server for every request (basket.UserId must equal the authenticated user's id), or derive the basket from the session token instead of accepting an identifier at all.

5.4 Finding 4 — Broken Access Control — Administrative Configuration Disclosure

Severity: High

CWE: CWE-862 — Missing Authorization

Location: GET /rest/admin/application-configuration

The administration page itself correctly refuses non-admin users (403) — but the API behind the admin area does not. As an ordinary registered user, the tester retrieved the complete application configuration: server settings, the chatbot's LLM model, the Google OAuth client ID, the product catalogue, security contacts, and challenge configuration. Authorization was enforced in the interface, not on the server.

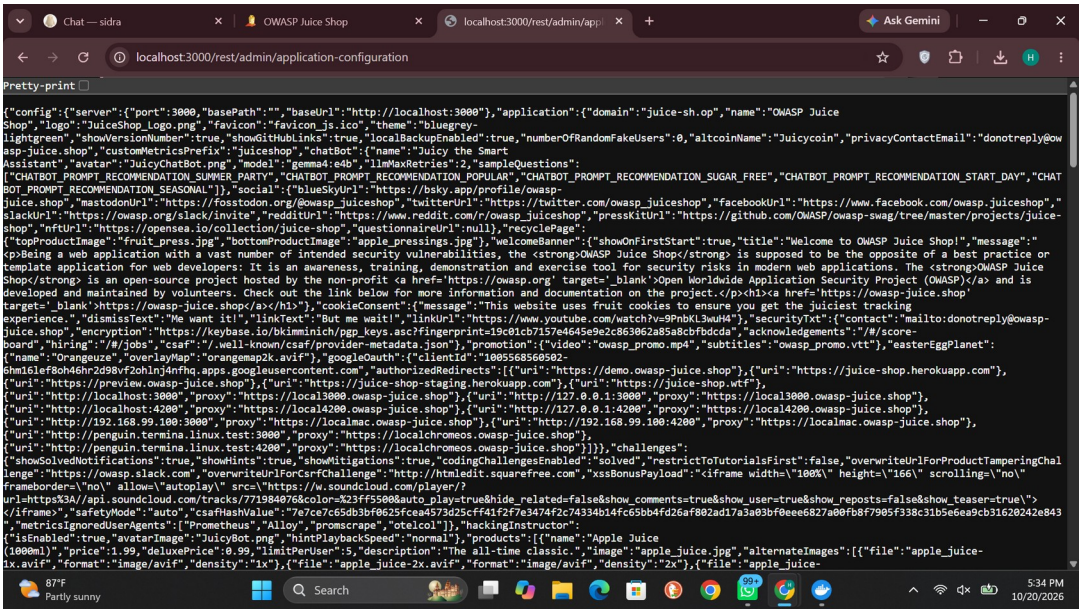


Figure 5 — F4: full application configuration JSON returned to a normal, non-admin user

Impact: Internal configuration disclosure to any registered user; the exposed details materially assist further attacks (targeted phishing, endpoint abuse, attack planning against disclosed components).

Remediation: Enforce authentication plus an administrator role check server-side on every /rest/admin/* route, and audit the full route table for endpoints protected only by being hidden.

5.5 Finding 5 — Sensitive Data Exposure — Public /ftp Directory

Severity: High

CWE: CWE-552 — Files or Directories Accessible to External Parties

Location: http://localhost:3000/ftp (directory listing, unauthenticated)

The /ftp path serves a browsable directory listing to anyone, with no authentication. Its contents include incident-support.kdbx (a KeePass password vault), acquisitions.md (confidential acquisition planning), backup files (coupons_2013.md.bak, package.json.bak, package-lock.json.bak), an encryption script (encrypt.pyc), and an internal error log (suspicious_errors.yml). Markdown files download directly; the scanner had already retrieved a confidential document from this directory during reconnaissance.

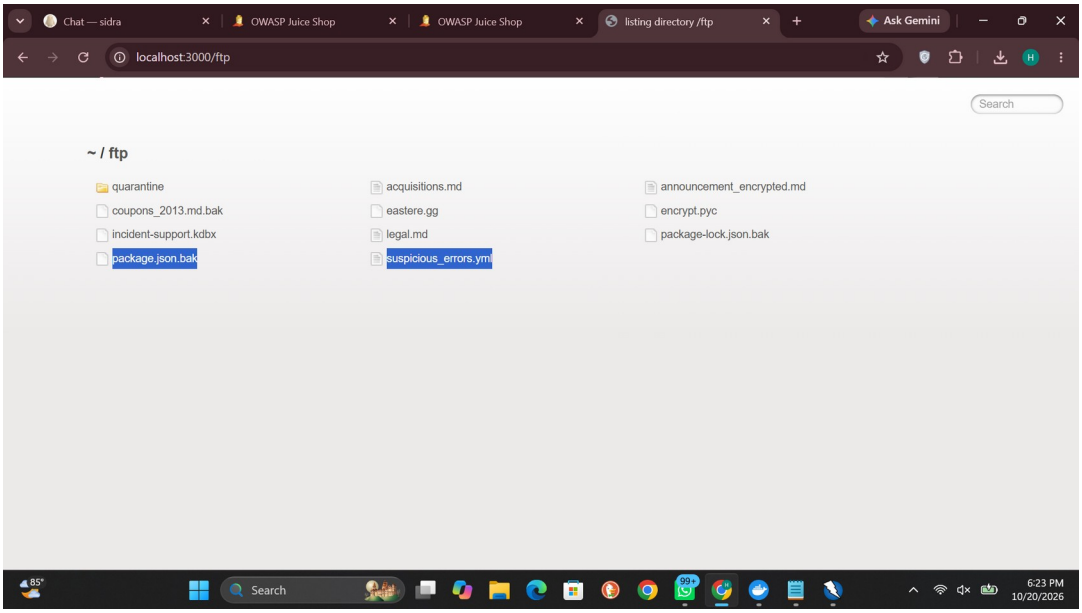


Figure 6 — F5: unauthenticated /ftp listing exposing a password vault, backups, and confidential documents

Impact: Direct exposure of credential material and confidential business information to unauthenticated attackers; a KeePass vault invites offline password attacks against every credential it protects.

Remediation: Remove the directory from the web root entirely; serve only intended public files through controlled routes with authorization; never place backups, vaults, or internal documents under a web-accessible path; add the path to scanner/recon checklists.

5.6 Finding 6 — Unauthenticated Metrics Endpoint

Severity: Medium

CWE: CWE-200 — Exposure of Sensitive Information to an Unauthorized Actor

Location: GET /metrics (Prometheus exposition, unauthenticated)

The application publishes its full Prometheus metrics without authentication: HTTP request volumes by status class (including error counts), file-upload statistics, LLM token counters, and startup timings annotated with internal task names (validateConfig, datacreator, cleanupFtpFolder), revealing internal structure and operational behaviour.

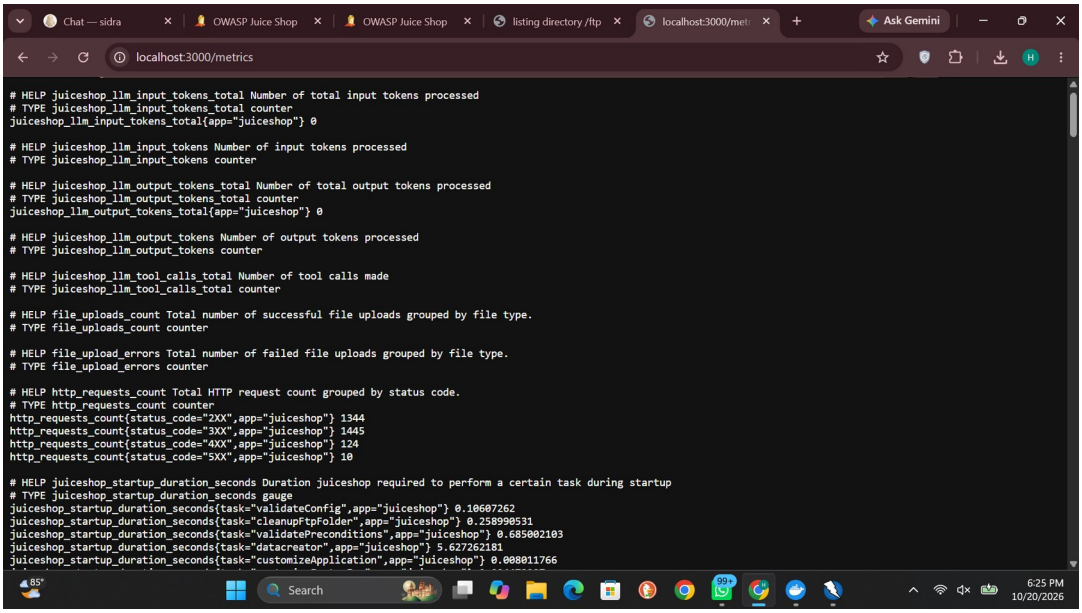


Figure 7 — F6: /metrics output — request statistics and internal startup task names served without authentication

Impact: Operational intelligence for an attacker: error-rate spikes reveal when attacks are working, feature counters reveal what to target, and internal naming aids further reconnaissance.

Remediation: Restrict /metrics to the monitoring network or require authentication; expose only aggregated, non-sensitive series publicly, if any.

5.7 Finding 7 — Missing Security Headers & Permissive CORS (scanner-verified)

Severity: Medium

CWE: CWE-693 — Protection Mechanism Failure; CWE-942 — Permissive Cross-domain Policy

Location: Site-wide (ZAP baseline: 8 alerts, incl. CSP Not Set, Cross-Domain Misconfiguration, Missing Anti-clickjacking Header, X-Content-Type-Options Missing)

The baseline ZAP scan reported no Content-Security-Policy anywhere, a wildcard Access-Control-Allow-Origin: *, and anti-clickjacking / content-type headers present on static files but absent from API responses. These were verified manually in DevTools. In the follow-up hardening exercise (Project 4), all four were remediated uniformly via an nginx reverse proxy and the fixes were confirmed by rescan — demonstrating that this class of finding is cheap to close once it is measured.

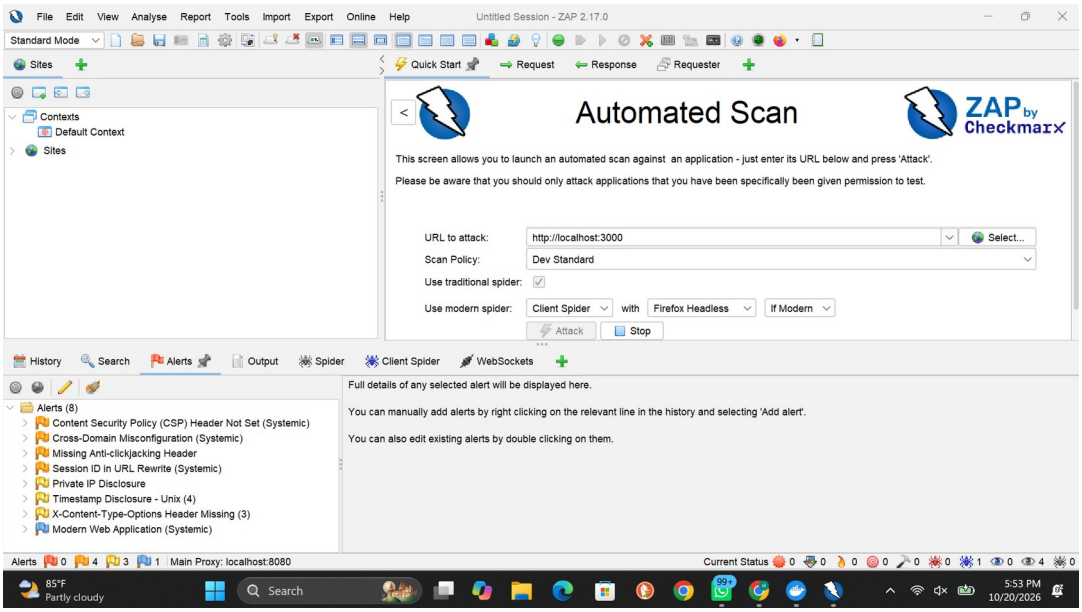


Figure 8 — F7: the ZAP baseline alerts against the unhardened application (Project 4)

Impact: Absence of CSP removes the safety net that limits XSS impact; the CORS wildcard lets any website read API responses cross-origin; inconsistent framing/sniffing headers leave clickjacking and MIME-confusion openings on API-driven views.

Remediation: Adopt the validated proxy header policy (CSP, X-Frame-Options SAMEORIGIN, X-Content-Type-Options nosniff, restricted CORS origin, Referrer-Policy) in production, and work toward a stricter nonce-based CSP.

6. Findings Summary & Attack Path

ID	Finding	Severity	Exploited / Retrieved
F1	SQL Injection — authentication bypass	Critical	Yes — administrator session
F2	DOM-Based XSS	High	Yes — script execution
F3	Basket IDOR	High	Yes — victim's basket read & editable
F4	Admin configuration disclosure (BAC)	High	Yes — full config as normal user
F5	Sensitive data exposure — /ftp directory	High	Yes — listing + confidential files
F6	Unauthenticated metrics endpoint	Medium	Yes — full metrics output
F7	Missing security headers & permissive CORS	Medium	Verified (scanner + manual); remediated via proxy in Project 4

Attack path (how the findings chain): an unauthenticated attacker starts with free intelligence — the /ftp vault and documents (F5), the metrics (F6), and, after self-registering, the full configuration (F4). The SQL injection (F1) then yields the administrator account outright, while the IDOR (F3) exposes every customer's basket without needing it, and the XSS (F2) turns ordinary customers into delivery vehicles against their own sessions. Individually serious, together they amount to full compromise of the application's data and its users — from a standing start, with no credentials and commodity tools.

7. Remediation Roadmap

Priority	Action	Addresses
Immediate	Parameterized queries everywhere; regression-test the login flow	F1
Immediate	Remove /ftp from the web root; rotate any credential stored in the exposed vault	F5
Immediate	Server-side authorization middleware on all /rest/admin/* and object routes (ownership checks)	F3, F4
Short term	Output-encoding audit of all DOM sinks; prohibit sanitizer bypasses in code review	F2
Short term	Adopt the validated security-header/CORS policy at the edge; plan nonce-based CSP	F7
Short term	Restrict /metrics to monitoring systems or authenticated access	F6
Strategic	Add the Broken Access Control checklist (Project 3, Annex A) and a ZAP baseline scan to the development pipeline so these classes are caught before release	All

8. Conclusion

The application fails its security assessment: seven confirmed findings, four of them exploitable to data or account compromise by attackers with little or no privilege. None of

the flaws is exotic — they are the OWASP classics (injection, XSS, broken access control, sensitive data exposure, missing hardening) — and none requires an advanced fix. The engagement's five projects form one continuous arc: map the target, break it three different ways, measure it with a scanner, harden it and prove the improvement, and finally report the whole assessment as a penetration test. The most transferable lesson is the one stated in the executive summary: every critical and high finding in this report exists because the application trusted something the client sent. Systems that stop doing that — binding input as data, encoding output, and checking authorization on the server for every request — do not have these findings.

Appendix A — Live Demo Runbook (10 minutes)

Preparation (before class): start Docker Desktop; confirm both containers are up (docker ps); open the shop at http://localhost:3000 and the hardened copy at http://localhost:4000 in separate tabs; keep the attacker account (attacker@test.com) in the main window and the victim account in an incognito window. If the lab was reset, recreate the two accounts and re-check their basket ids (bid) in DevTools → Application → Session storage — the IDOR step depends on those two numbers.

Time	Segment	What to show / say
0:00–1:00	Framing	Target, lab-only scope, and the headline: 7 findings, 1 Critical — all exploited live in the lab.
1:00–2:30	Recon pays off	Show the attack-surface map (Project 1), then open /ftp live: a password vault and confidential files with no login (F5).
2:30–4:30	Exploit 1 — SQLi	Log out; at the login form enter ' or 1=1-- as the email with any password → administrator session (F1). Explain: 1=1 is always true, -- deletes the password check.
4:30–6:00	Exploit 2 — XSS	In search, enter <iframe src="javascript:alert('xss')"> → alert fires (F2). Note why <script> fails but the iframe bypass works.
6:00–8:00	Exploit 3 — IDOR	In the attacker window, DevTools → Session storage: change bid to the victim's value, reload, open Your Basket → the victim's items under the attacker's name (F3).

8:00–9:00	Access control + exposure	Open /rest/admin/application-configuration as a normal user (F4) and /metrics (F6) — the page is protected, the data is not.
9:00–10:00	Fixes exist	Switch to http://localhost:4000 and show the same response headers in DevTools: CSP present, CORS restricted (Project 4). Close with the remediation roadmap.

Fallbacks: if any live step misbehaves, the matching screenshot in Section 5 tells the same story — switch to it without stopping; the demo is evidence-backed, not luck-based.

Appendix B — Evidence Index

Figure	Content	Finding
1	Target application product catalogue	Recon
2	SQLi payload as username + CHALLENGE_SOLVED (admin)	F1
3	XSS alert executed, payload in URL, challenge solved	F2
4	Victim's basket under attacker's account	F3
5	Full application configuration as normal user	F4
6	/ftp directory listing (vault, backups, documents)	F5
7	/metrics Prometheus output	F6
8	ZAP baseline alerts (8) against :3000	F7

Detailed per-project evidence and working notes are in the companion reports: Project 1 (Map the Target), Project 2 (Injection Hunter), Project 3 (Broken Access Control Audit), and Project 4 (Harden and Rescan).